

HW1_Problem1

January 28, 2026

1 Problem 1

```
[16]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
from sklearn.linear_model import LinearRegression
from pathlib import Path
import zipfile
import urllib.request
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error,
↳mean_absolute_percentage_error, mean_squared_error, accuracy_score
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import train_test_split, cross_val_score, KFold
```

1.1 1.1

1.1.1 Open a Jupyter-notebook. Download the wines dataset and load the data of the “red” wines.

1.2 Fetch & Load Wine_Quality_Data

```
[17]: from pathlib import Path
import pandas as pd

def load_wine_quality_data():
    '''This function downloads and unzips the wine_quality.zip file,
    containing the winequality-white.csv and winequality-red.csv. After the_
    ↳extraction,
    the function returns the winequality-red.csv as a Pandas dataframe'''

    zip_path = Path('datasets/wine_quality.zip') # Path to the ZIP file
    dataset_folder = Path('datasets/wine_quality_dataset') # Path to the_
    ↳extracted dataset folder
```

```

    if not dataset_folder.exists(): # Check if dataset is already downloaded
        Path('datasets').mkdir(parents=True, exist_ok=True) # In case it
        ↪ doesn't exist, create the datasets folder

    url = 'https://archive.ics.uci.edu/static/public/186/wine+quality.zip' ↪
    ↪ # URL of the dataset
    urllib.request.urlretrieve(url, zip_path) # Download the ZIP file to
    ↪ the path given earlier

    with zipfile.ZipFile(zip_path) as zip_ref: # Open the ZIP file
        zip_ref.extractall(dataset_folder) # Extract all files to the
        ↪ dataset folder
    print('Dataset downloaded and extracted successfully.')

    return pd.read_csv(dataset_folder / 'winequality-red.csv') # Return the
    ↪ DataFramea
wine_quality_data = load_wine_quality_data()

red_wine_df = pd.read_csv('datasets/wine_quality_dataset/winequality-red.csv',
    ↪ sep=';')

```

Print the dataset's first rows

```
[18]: red_wine_df.head()
```

```

[18]:   fixed acidity  volatile acidity  citric acid  residual sugar  chlorides \
0           7.4           0.70           0.00           1.9           0.076
1           7.8           0.88           0.00           2.6           0.098
2           7.8           0.76           0.04           2.3           0.092
3          11.2           0.28           0.56           1.9           0.075
4           7.4           0.70           0.00           1.9           0.076

      free sulfur dioxide  total sulfur dioxide  density  pH  sulphates \
0              11.0           34.0  0.9978  3.51           0.56
1              25.0           67.0  0.9968  3.20           0.68
2              15.0           54.0  0.9970  3.26           0.65
3              17.0           60.0  0.9980  3.16           0.58
4              11.0           34.0  0.9978  3.51           0.56

      alcohol  quality
0          9.4         5
1          9.8         5
2          9.8         5
3          9.8         6
4          9.4         5

```

1.3 1.2

1.3.1 What are the features describing the quality of the wines?

Overview of the red wine dataframe

```
[19]: red_wine_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   fixed acidity          1599 non-null   float64
1   volatile acidity       1599 non-null   float64
2   citric acid            1599 non-null   float64
3   residual sugar         1599 non-null   float64
4   chlorides              1599 non-null   float64
5   free sulfur dioxide    1599 non-null   float64
6   total sulfur dioxide   1599 non-null   float64
7   density                1599 non-null   float64
8   pH                    1599 non-null   float64
9   sulphates              1599 non-null   float64
10  alcohol                1599 non-null   float64
11  quality                1599 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 150.0 KB
```

We notice there are 12 different features, 1599 non-null values out of 1599 entries, which means there are no missing values. 11 columns are of type float64 and describe continuous numerical features, while 1 column is of type int64, and describes a discrete numerical feature.

Let's store the feature names in a list and print them out.

```
[20]: red_wine_df_col = list(red_wine_df.columns) # Get the columns
print(f'The features of the dataset are: {", ".join(red_wine_df_col)}.') # Join
    ↪ them
```

The features of the dataset are: fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol, quality.

1.4 1.3

1.4.1 Compute the descriptive statistics of the dataset features and discuss about their types, ranges and completeness.

Check the dataset features' descriptive statistics

```
[21]: red_wine_df.describe()
```

```
[21]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	\
count	1599.000000	1599.000000	1599.000000	1599.000000	
mean	8.319637	0.527821	0.270976	2.538806	
std	1.741096	0.179060	0.194801	1.409928	
min	4.600000	0.120000	0.000000	0.900000	
25%	7.100000	0.390000	0.090000	1.900000	
50%	7.900000	0.520000	0.260000	2.200000	
75%	9.200000	0.640000	0.420000	2.600000	
max	15.900000	1.580000	1.000000	15.500000	

	chlorides	free sulfur dioxide	total sulfur dioxide	density	\
count	1599.000000	1599.000000	1599.000000	1599.000000	
mean	0.087467	15.874922	46.467792	0.996747	
std	0.047065	10.460157	32.895324	0.001887	
min	0.012000	1.000000	6.000000	0.990070	
25%	0.070000	7.000000	22.000000	0.995600	
50%	0.079000	14.000000	38.000000	0.996750	
75%	0.090000	21.000000	62.000000	0.997835	
max	0.611000	72.000000	289.000000	1.003690	

	pH	sulphates	alcohol	quality
count	1599.000000	1599.000000	1599.000000	1599.000000
mean	3.311113	0.658149	10.422983	5.636023
std	0.154386	0.169507	1.065668	0.807569
min	2.740000	0.330000	8.400000	3.000000
25%	3.210000	0.550000	9.500000	5.000000
50%	3.310000	0.620000	10.200000	6.000000
75%	3.400000	0.730000	11.100000	6.000000
max	4.010000	2.000000	14.900000	8.000000

Inspect the features data types and completeness

```
[22]: red_wine_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   fixed acidity          1599 non-null   float64
1   volatile acidity       1599 non-null   float64
2   citric acid            1599 non-null   float64
3   residual sugar         1599 non-null   float64
4   chlorides              1599 non-null   float64
5   free sulfur dioxide    1599 non-null   float64
6   total sulfur dioxide   1599 non-null   float64
7   density                1599 non-null   float64
8   pH                    1599 non-null   float64
```

```

9   sulphates          1599 non-null   float64
10  alcohol            1599 non-null   float64
11  quality            1599 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 150.0 KB

```

```
[23]: red_wine_df.isnull().sum()
```

```

[23]: fixed acidity      0
      volatile acidity  0
      citric acid       0
      residual sugar    0
      chlorides         0
      free sulfur dioxide 0
      total sulfur dioxide 0
      density          0
      pH              0
      sulphates        0
      alcohol          0
      quality          0
      dtype: int64

```

```

[24]: def feature_type(column_name):
      '''
      Determines the type of a given feature.
      '''
      if column_name == 'quality':
          return 'Ordinal Categorical' # Checks if the column name is 'quality',
      ↪ which is of ordinal categorical type.
      else:
          return 'Continuous Numerical' # Every other column is of continuous
      ↪ numerical type.

```

Present the type, range and completeness of the dataset features

```

[25]: descriptive_stats = red_wine_df.describe()

for column in descriptive_stats.columns: # Loop through the columns to retrieve
    ↪ its descriptive statistics and store their type, min and max values, and
    ↪ count.
    f_type = feature_type(column)
    min_val = descriptive_stats.loc['min', column]
    max_val = descriptive_stats.loc['max', column]
    count_val = descriptive_stats.loc['count', column]

    # Print the features' details
    print(f'Feature: {column}')
    print(f' - Type: {f_type}')

```

```

print(f' - Range: [{min_val}, {max_val}]')
print(f' - Completeness: {int(count_val)} / {len(red_wine_df)} values_
present')
print('-' * 40) # Separator

```

Feature: fixed acidity

- Type: Continuous Numerical
- Range: [4.6, 15.9]
- Completeness: 1599 / 1599 values present

Feature: volatile acidity

- Type: Continuous Numerical
- Range: [0.12, 1.58]
- Completeness: 1599 / 1599 values present

Feature: citric acid

- Type: Continuous Numerical
- Range: [0.0, 1.0]
- Completeness: 1599 / 1599 values present

Feature: residual sugar

- Type: Continuous Numerical
- Range: [0.9, 15.5]
- Completeness: 1599 / 1599 values present

Feature: chlorides

- Type: Continuous Numerical
- Range: [0.012, 0.611]
- Completeness: 1599 / 1599 values present

Feature: free sulfur dioxide

- Type: Continuous Numerical
- Range: [1.0, 72.0]
- Completeness: 1599 / 1599 values present

Feature: total sulfur dioxide

- Type: Continuous Numerical
- Range: [6.0, 289.0]
- Completeness: 1599 / 1599 values present

Feature: density

- Type: Continuous Numerical
- Range: [0.99007, 1.00369]
- Completeness: 1599 / 1599 values present

Feature: pH

- Type: Continuous Numerical
- Range: [2.74, 4.01]

- Completeness: 1599 / 1599 values present

Feature: sulphates

- Type: Continuous Numerical
- Range: [0.33, 2.0]
- Completeness: 1599 / 1599 values present

Feature: alcohol

- Type: Continuous Numerical
- Range: [8.4, 14.9]
- Completeness: 1599 / 1599 values present

Feature: quality

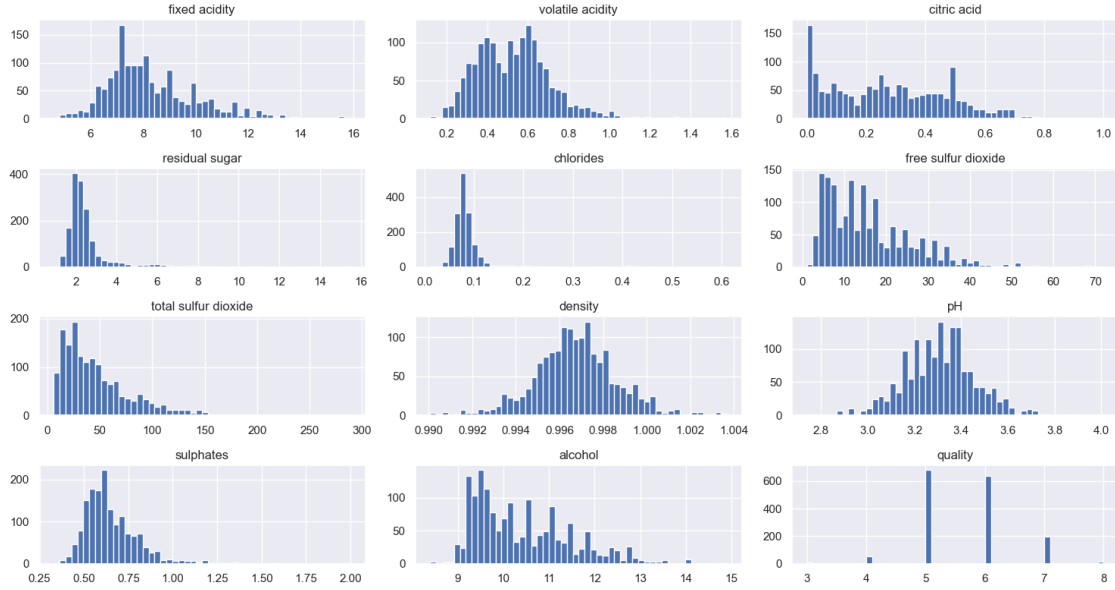
- Type: Ordinal Categorical
- Range: [3.0, 8.0]
- Completeness: 1599 / 1599 values present

Domain knowledge observations: The wines in this dataset represent a diverse range. Most of the wines fall within expected ranges for acidity, alcohol content, and pH. A wide range of values for Residual Sugar, shows that the wines include both dry and sweet wines. The quality ratings indicate that most wines are average, with no wines rated extremely poorly (min rating is 3) or excellently (max rating is 8).

1.5 1.4

1.5.1 Form the histograms of the features and discuss their distribution. Can the distribution of some features be improved (tending more towards the Gaussian) and how?

```
[26]: red_wine_df.hist(bins=50, figsize=(15,8)) # Create the histograms, with 50 bins
plt.tight_layout() # Use tight_layout so that the histogram title does not
↳ collide with the x-axis ticks/values.
plt.show()
```



Setting aside the Quality feature, which is a categorical variable with distinct values, we observe that most distributions are right-skewed. Others have a strong right skew (Total Sulfur Dioxide, Chlorides, Residual Sugar) while others have a slight right skew (Alcohol, Fixed Acidity, Volatile Acidity). We also notice that Density and pH are nearly symmetrical, close to normal distribution. For the right-skewed features, we should apply a logarithmic transformation to reduce the skewness and make their distributions closer to Gaussian followed by scaling techniques such as standardization, to achieve a mean of 0 and a standard deviation of 1, or min-max scaling to bring the values into a fixed range, like $[0,1]$ or $[-1,1]$.

1.6 1.5

1.6.1 Which are the features that mostly affect quality and which are those that affect it less? Provide evidence through correlation and discuss accordingly.

```
[27]: red_wine_df.corr()
```

```
[27]:
```

	fixed acidity	volatile acidity	citric acid	\
fixed acidity	1.000000	-0.256131	0.671703	
volatile acidity	-0.256131	1.000000	-0.552496	
citric acid	0.671703	-0.552496	1.000000	
residual sugar	0.114777	0.001918	0.143577	
chlorides	0.093705	0.061298	0.203823	
free sulfur dioxide	-0.153794	-0.010504	-0.060978	
total sulfur dioxide	-0.113181	0.076470	0.035533	
density	0.668047	0.022026	0.364947	
pH	-0.682978	0.234937	-0.541904	
sulphates	0.183006	-0.260987	0.312770	
alcohol	-0.061668	-0.202288	0.109903	

quality	0.124052	-0.390558	0.226373
---------	----------	-----------	----------

	residual sugar	chlorides	free sulfur dioxide	\
fixed acidity	0.114777	0.093705	-0.153794	
volatile acidity	0.001918	0.061298	-0.010504	
citric acid	0.143577	0.203823	-0.060978	
residual sugar	1.000000	0.055610	0.187049	
chlorides	0.055610	1.000000	0.005562	
free sulfur dioxide	0.187049	0.005562	1.000000	
total sulfur dioxide	0.203028	0.047400	0.667666	
density	0.355283	0.200632	-0.021946	
pH	-0.085652	-0.265026	0.070377	
sulphates	0.005527	0.371260	0.051658	
alcohol	0.042075	-0.221141	-0.069408	
quality	0.013732	-0.128907	-0.050656	

	total sulfur dioxide	density	pH	sulphates	\
fixed acidity	-0.113181	0.668047	-0.682978	0.183006	
volatile acidity	0.076470	0.022026	0.234937	-0.260987	
citric acid	0.035533	0.364947	-0.541904	0.312770	
residual sugar	0.203028	0.355283	-0.085652	0.005527	
chlorides	0.047400	0.200632	-0.265026	0.371260	
free sulfur dioxide	0.667666	-0.021946	0.070377	0.051658	
total sulfur dioxide	1.000000	0.071269	-0.066495	0.042947	
density	0.071269	1.000000	-0.341699	0.148506	
pH	-0.066495	-0.341699	1.000000	-0.196648	
sulphates	0.042947	0.148506	-0.196648	1.000000	
alcohol	-0.205654	-0.496180	0.205633	0.093595	
quality	-0.185100	-0.174919	-0.057731	0.251397	

	alcohol	quality
fixed acidity	-0.061668	0.124052
volatile acidity	-0.202288	-0.390558
citric acid	0.109903	0.226373
residual sugar	0.042075	0.013732
chlorides	-0.221141	-0.128907
free sulfur dioxide	-0.069408	-0.050656
total sulfur dioxide	-0.205654	-0.185100
density	-0.496180	-0.174919
pH	0.205633	-0.057731
sulphates	0.093595	0.251397
alcohol	1.000000	0.476166
quality	0.476166	1.000000

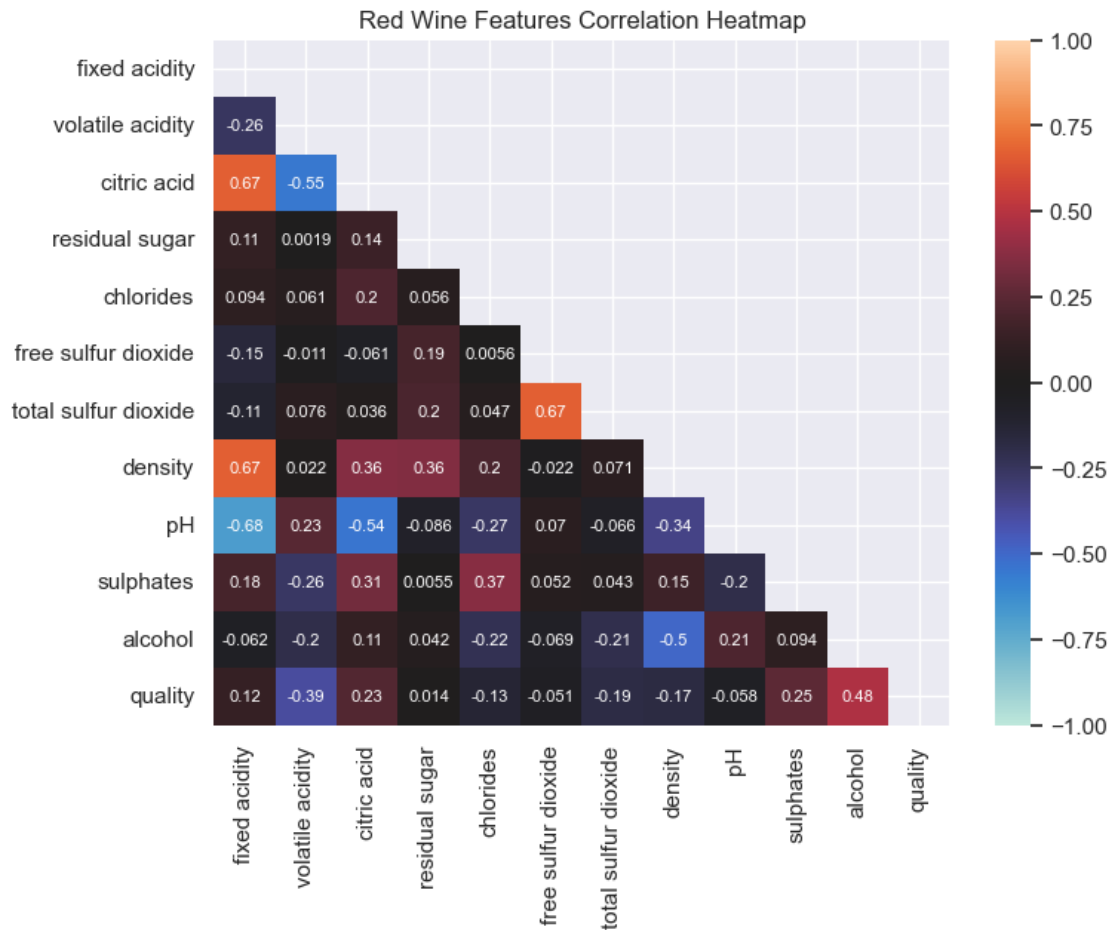
```
[28]: corr_matrix = red_wine_df.corr()
corr_matrix['quality'].sort_values(ascending=False)
```

```
[28]: quality          1.000000
      alcohol         0.476166
      sulphates       0.251397
      citric acid      0.226373
      fixed acidity    0.124052
      residual sugar   0.013732
      free sulfur dioxide -0.050656
      pH              -0.057731
      chlorides        -0.128907
      density          -0.174919
      total sulfur dioxide -0.185100
      volatile acidity  -0.390558
      Name: quality, dtype: float64
```

Create the heatmap with mask applied

```
[29]: mask = np.triu(np.ones_like(red_wine_df.corr(), dtype=bool)) # Create a mask
      ↪for the upper triangle

      plt.figure(figsize=(8, 6))
      sns.heatmap(red_wine_df.corr(), annot=True, cmap='icefire', vmin=-1, vmax=1,
      ↪mask=mask, annot_kws={"size": 8})
      plt.title('Red Wine Features Correlation Heatmap')
      plt.show()
```



1.7 1.6

1.7.1 Split the dataset into a training and a testing set retaining 80% and 20% of the total number of samples, respectively, using random shuffling and splitting that retains the statistical properties of the input data (stratified) with respect to quality.

Split the dataset into training and test sets, with a ratio of 80%-20%. I use stratified sampling to ensure that the distribution of the 'quality' target variable, is retained in the new sets.

```
[30]: strat_train_set, strat_test_set = train_test_split(red_wine_df, test_size=0.2,
↳stratify=red_wine_df['quality'], random_state=42)
```

Confirm with manual calculations, that the stratification maintained a similar distribution. Pass the results into a dataframe and print them.

```
[31]: test_strat_ratio = (strat_test_set['quality'].value_counts() /
↳len(strat_test_set)).sort_index()
```

```

train_strat_ratio = (strat_train_set['quality'].value_counts() /
    ↪len(strat_train_set)).sort_index()
original_strat_ratio = (red_wine_df['quality'].value_counts() /
    ↪len(red_wine_df)).sort_index()

ratios_df = pd.DataFrame({
    'Original %': (original_strat_ratio * 100).round(2),
    'Train %': (train_strat_ratio * 100).round(2),
    'Test %': (test_strat_ratio * 100).round(2)
})

```

```
[32]: ratios_df
```

```

[32]:          Original %  Train %  Test %
quality
3           0.63      0.63    0.62
4           3.31      3.28    3.44
5          42.59     42.61   42.50
6          39.90     39.87   40.00
7          12.45     12.43   12.50
8           1.13      1.17    0.94

```

The quality distribution is retained in both training and test set. This was expected, since we used stratified splitting which ensures exactly that.

1.8 1.7

1.8.1 Scale the data with a Standard scaler and train a linear regression model. Evaluate the performance of the model, using the test set, with respect to metrics such as R2 -score, Mean Absolute Error, Mean Absolute Percentage Error, Mean Squared Error and Accuracy. Comment on the accuracy of predictions by plotting Actuals vs Predicted diagram.

Reset the indices for the train and the test set. When splitting datasets, the indices often retain their original values, which might be scattered.

Resetting indices ensures clean, sequential indices (0, 1, 2, ...) for both datasets.

It avoids errors later, especially when combining predictions or debugging.

```

[33]: strat_train_set.reset_index(drop=True, inplace=True)
      strat_test_set.reset_index(drop=True, inplace=True)

```

To train the model, we need to separate:

X: Independent variables (features that describe the wine).

y: Dependent variable (wine quality in this case).

This is a standard practice in supervised machine learning problems

```

[34]: X_train = strat_train_set.drop('quality',axis=1)
      y_train = strat_train_set['quality']

```

```
X_test = strat_test_set.drop('quality', axis=1)
y_test = strat_test_set['quality']
```

Many machine learning models, including Linear Regression, perform better when features are standardized.

StandardScaler transforms features to have mean = 0 and standard deviation = 1.

Important: The scaler is fitted only on the training set, to prevent data leakage. The same scaler is then applied to the test set.

```
[35]: target_scaler = StandardScaler()

X_train_scaled = target_scaler.fit_transform(X_train)
X_test_scaled = target_scaler.transform(X_test)
```

Initialize the Linear Regression model, using the scaled training features and the training target variable. Then, use the trained model to predict the ‘quality’ of the wines in the test set. It’s a simple baseline regression model.

It tries to find the best-fitting line by minimizing the Mean Squared Error (MSE) between actual and predicted values.

It’s a good starting point, but not always the best for complex relationships.

The model, after training, is now used to make predictions on unseen data (X_test_scaled).

Outputs are continuous values, as Linear Regression predicts floats.

```
[36]: model = LinearRegression()
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
```

Calculate the requested metrics, to evaluate the performance of our model. R² Score (Coefficient of Determination): Measures how much variance in the target variable is explained by the model.

Value of 1 means perfect prediction.

Value of 0 means the model is as good as the mean of y.

MAE (Mean Absolute Error): Average absolute error. Easy to interpret as it’s in the same units as quality.

MAPE (Mean Absolute Percentage Error): Measures the error as a percentage. Useful to understand errors relative to the actual values.

MSE (Mean Squared Error): Penalizes larger errors more heavily (squared).

```
[37]: # R2 Score
r2 = r2_score(y_test, y_pred)

# Mean Absolute Error
mae = mean_absolute_error(y_test, y_pred)

# Mean Absolute Percentage Error
mape = mean_absolute_percentage_error(y_test, y_pred)
```

```
# Mean Squared Error
mse = mean_squared_error(y_test, y_pred)
```

The target variable ‘quality’ in the dataset, takes integer values in range [3, 8]. The linear regression model, outputs continuous values, that is float numbers. Since the actual wine quality is always an integer, we round those predictions, and then we keep only the ones that fall in the valid range of our dataset’s wine quality. Accuracy measures how well the model performed in predicting the actual quality values. Had we not rounded the predictions, it would be virtually impossible to achieve an exact match and we would get a misleadingly low evaluation score.

```
[38]: y_pred_rounded = np rint(y_pred) # Round predictions to the nearest integer

y_pred_rounded = np.clip(y_pred_rounded, y_test.min(), y_test.max()) # Ensure
↳ predictions are within the valid 'quality' range

accuracy = accuracy_score(y_test, y_pred_rounded)
```

Create a dictionary for the metrics and pass it into a dataframe.

```
[39]: metrics = {'Metric': ['R2 Score', 'Mean Absolute Error (MAE)', 'Mean Absolute
↳ Pct Error (MPAE)', 'Mean Squared Error (MSE)', 'Accuracy'],
                 'Value': [round(r2,3), round(mae,3), round(mape,3), round(mse,3),
↳ round(accuracy,3)]}
```

```
[40]: metrics_df = pd.DataFrame(metrics)
metrics_df
```

```
[40]:
```

	Metric	Value
0	R ² Score	0.370
1	Mean Absolute Error (MAE)	0.495
2	Mean Absolute Pct Error (MPAE)	0.091
3	Mean Squared Error (MSE)	0.406
4	Accuracy	0.597

The model’s R² score of 0.37, suggests that only 37% of the variance in the ‘quality’ scores is being explained by the model and the model’s Accuracy score of 0.597, suggests that the model correctly predicts the wine quality around 59.7% of the time. Our model is not performing particularly well, which might mean that a linear regression model might not be the best fit for this problem.

This can also be seen in the visualization below.

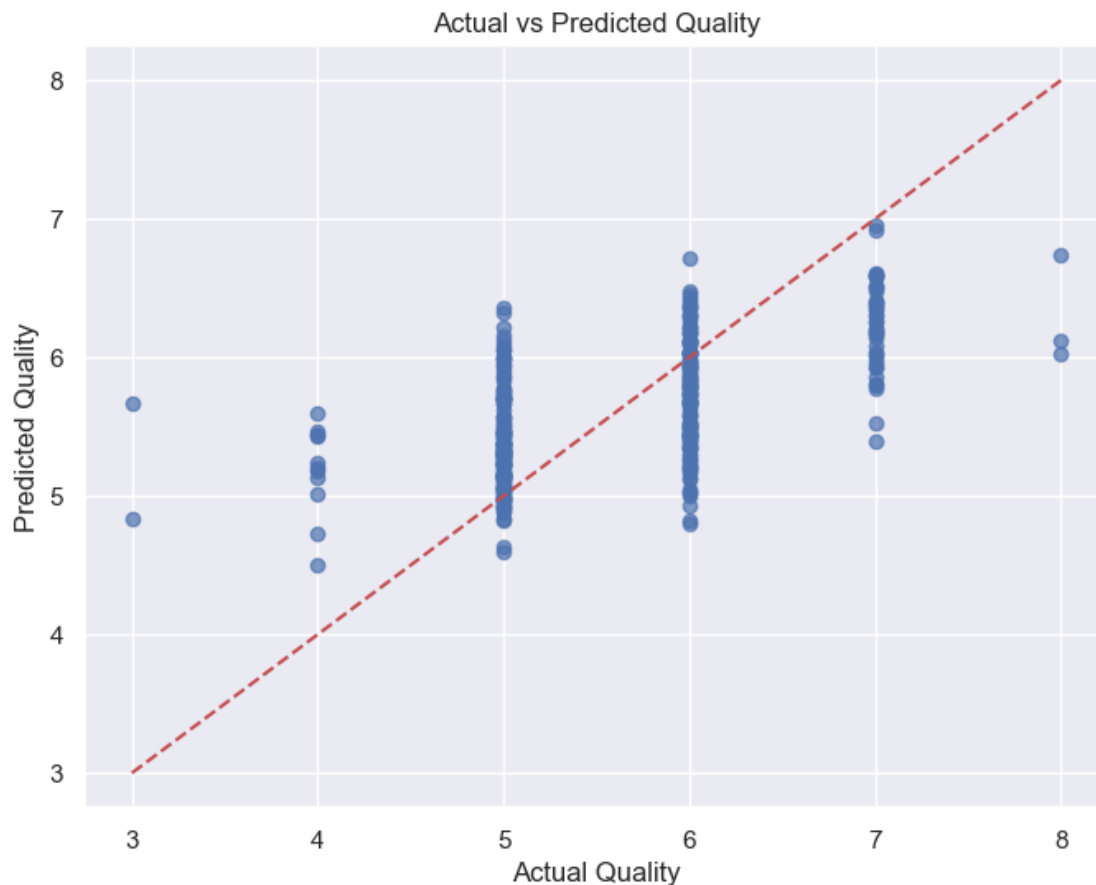
Plot Actual vs Predicted values

```
[41]: plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.7, color='b')
plt.xlabel('Actual Quality')
plt.ylabel('Predicted Quality')
plt.title('Actual vs Predicted Quality')
```

```

diagonal = np.linspace(y_test.min(), y_test.max(), 100) # Generate 100 values
↳ between the minimum value of the y_test and
                                                    # the maximum value of
↳ y_test
plt.plot(diagonal, diagonal, 'r--') # Draw a diagonal line, where x and y are
↳ equal, hence the y=x line. That represents a
                                                    # diagonal reference line that indicates
↳ the perfect predictions.
plt.show()

```



We observe that the points fall far off from the reference prediction reference line.

1.9 1.8

1.9.1 Perform 10-fold cross validation and compute the mean and standard deviation of the scores over the folds. Is the model's R2-score within the limits defined by the 10-fold cross validation?

Initialize the linear regression model, and define the 10-fold CV using KFold. Perform cross validation, computing the R^2 for each fold, and store the results in cv_scores.

The training set is split into 10 parts.

The model is trained on 9 parts and validated on the 10th.

This repeats 10 times, so every fold gets to be the validation set once

The R^2 score is computed in each fold and saved in `cv_scores`.

```
[42]: linear_model = LinearRegression()

kf = KFold(n_splits=10, shuffle=True, random_state=42)

cv_scores = cross_val_score(linear_model, X_train_scaled, y_train, cv=kf,
                             scoring='r2')
print("Cross-Validation  $R^2$  Scores:")
print(cv_scores)
```

Cross-Validation R^2 Scores:

```
[0.43091694 0.29793909 0.41938328 0.27331003 0.39947422 0.19204422
 0.38719947 0.25085842 0.34651846 0.26805618]
```

Calculate the mean and standard deviation of R^2 scores

Mean $R^2 \rightarrow$ Average performance of the model across the folds.

Standard Deviation of $R^2 \rightarrow$ Measures stability and variance of the model's performance across folds.

This gives you an idea of how much the model's performance fluctuates across different training-validation splits.

```
[43]: mean_r2 = cv_scores.mean()
std_r2 = cv_scores.std()

print(f"\nMean  $R^2$  Score: {mean_r2:.4f}")
print(f"Standard Deviation of  $R^2$  Scores: {std_r2:.4f}")
```

Mean R^2 Score: 0.3266

Standard Deviation of R^2 Scores: 0.0773

Now that CV is done, we train the model on the entire training set to prepare for final evaluation on the test set.

```
[44]: linear_model.fit(X_train_scaled, y_train)
```

```
[44]: LinearRegression()
```

Predict the target variable for the test data

```
[45]: y_pred = linear_model.predict(X_test_scaled)
```

Calculate R^2 score on the test set. We evaluate the trained model on unseen data.

```
[46]: test_r2 = r2_score(y_test, y_pred)
```

```
print(f"\nTest Set R2 Score: {test_r2:.4f}")
```

Test Set R² Score: 0.3703

Calculate the limits

We calculate a range of “expected” R² scores based on CV:

Lower Limit = Mean R² - Std R²

Upper Limit = Mean R² + Std R²

This defines a confidence interval-like band to judge if the model generalizes as expected.

```
[47]: lower_limit = mean_r2 - std_r2
      upper_limit = mean_r2 + std_r2

      print(f"\nCross-Validation R2 Score Range: [{lower_limit:.4f}, {upper_limit:.4f}]" )
```

Cross-Validation R² Score Range: [0.2492, 0.4039]

Check if test R² score is within the limits

```
[48]: if lower_limit <= test_r2 <= upper_limit:
      print(f"The model's test R2 score: {test_r2:.4f} is within the limits_
      ↪defined by the 10-fold cross-validation.")
      else:
      print(f"The model's test R2 score: {test_r2:.4f} is NOT within the limits_
      ↪defined by the 10-fold cross-validation.")
```

The model's test R² score: 0.3703 is within the limits defined by the 10-fold cross-validation.